MANUAL PID TUNING OF DC MOTOR ANGLE CONTROL

1.0 Learning Outcomes.....1
2.0 Hardware Set-Up.....1
3.0 Practical Investigation via Simulink.....1
4.0 Concluding Remarks.....4
Recommended Citation.....4

1.0 Learning Outcomes

Upon completing this practical exercise, students should be able to demonstrate the Learning Outcomes presented in Table 1. At the end of the exercise, revisit each Learning Outcome and confirm that it has been achieved.

Table 1: *Learning Outcomes for the Practical Exercise*

ID	Learning Outcome	Achieved
1	Measure the angle of the DC motor shaft from the encoder and close a PID position loop around it in Simulink.	☐
2	Tune the proportional, derivative and integral gains in turn from the measured response, linking each change to the rise time, overshoot and steady-state error.	☐
3	Assess the robustness of the tuned controller by adding a load to the shaft and repeating the test.	☐

2.0 Hardware Set-Up

This practical exercise builds on the hardware and algorithm set-up described in the practical exercise, DC Motor Speed Measurement using an Encoder, which details the operation of a DC motor with an encoder. In this exercise the encoder counts are used directly to measure the angle of the shaft rather than its speed.

Fix a light pointer, such as a strip of card, to the shaft, and place a paper dial marked every 30° behind it so that the angle can be checked by eye. For the robustness test, a small mass such as a coin can be taped to the end of the pointer.

3.0 Practical Investigation via Simulink

MATLAB and Simulink should be used to complete this practical exercise. A PID controller is used to move the shaft to a commanded angle. The gains are tuned by hand, one at a time, using the measured response to decide each change. The exercise extends the speed controller tuning in the practical exercise, DC Motor PI Control: Manual Tuning, to position control, where the derivative term becomes important. It supports the ACE-CORE learning guide, Manual Tuning of a PID Controller.

Background: Angle Measurement and PID Control

The encoder produces a fixed number of counts per revolution of the output shaft, *CPR*. The angle in degrees follows from the accumulated count *n*:

$$\theta = \frac{360^\circ \times n}{CPR}$$

The angle is the integral of the speed, so even proportional control alone gives a second-order closed loop that can overshoot. The derivative term adds damping. It is calculated from the measured angle rather than the error, so that a step in the setpoint does not cause a spike in the output, and it is filtered to limit the effect of encoder quantisation:

$$u[k] = K_p\, e[k] + K_i\, T_s \sum_{j=0}^{k} e[j] - K_d\, \frac{\theta_f[k] - \theta_f[k-1]}{T_s}$$

where θ_f is the filtered angle. The output *u* is a voltage between −5 V and +5 V, whose sign sets the direction of the H-bridge and whose magnitude sets the PWM value.

[Screenshot to insert] *Step (angle setpoint, deg) and encoder count → angle (360·n/CPR) → PID with filtered derivative → saturation ±5 V → sign to direction pin, |u|·255/5 to PWM. Scopes log r, θ, u and t.*

Figure 1: *Simulink Model for PID Control of the Motor Angle*

1. Build the model in Figure 1 with T_s = 0.01 s. Use a PID Controller block in discrete time, or a MATLAB Function block implementing the equation above, with a derivative filter coefficient giving a filter time constant of about 0.02 s.
2. Set the setpoint to step from 0° to 90° at *t* = 1 s, and log the setpoint, the angle, the output voltage and the time.
3. Check the sign of the loop first with a small proportional gain: the shaft must move towards the setpoint. If it moves away, swap the motor wires or change the sign of the output.

Tune the Proportional Gain

1. Start with K_p = 0.01 V/° and K_i = K_d = 0. Run the step and record the rise time, overshoot and final error in Table 2.
2. Double K_p after each run until the overshoot reaches about 20 to 30%.
3. Note any final error. With proportional control alone, friction can stop the shaft just short of the setpoint, because a small error gives too little voltage to overcome it.

Add Derivative Action

1. Keeping the last K_p, set K_d = 0.001 V·s/° and increase it in steps until the overshoot falls below about 5%.
2. Listen to the motor and look at the output voltage. If the motor buzzes or the voltage is very noisy, the derivative gain is amplifying encoder noise: reduce K_d or slow the derivative filter.

Example responses are shown in Figure 2. They are simulated with the motor model identified in the practical exercise, Parameter Estimation of a DC Motor using Linear Least Squares (K = 28 RPM/V, τ = 0.15 s), so they show the behaviour to expect rather than measured data, integrated to give the angle. The simulation has no friction, so it shows no final error with proportional control alone. Your motor will usually stop slightly short of the setpoint.

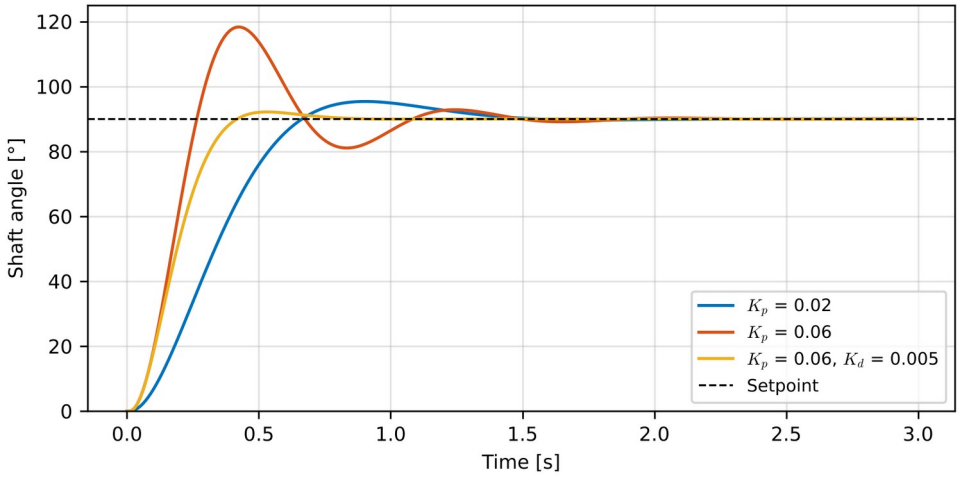


Figure 2: *Simulated Example of the Angle Response to a 90° Step for Proportional and PD Control*

Add Integral Action

1. Increase K_i slowly from zero until any remaining final error is removed within about 2 s.
2. If the shaft starts to move slowly back and forth around the setpoint (hunting), reduce K_i.
3. For your final gains, measure the rise time, overshoot, settling time and final error, and record them in Table 2.
4. Tape the small mass to the pointer and repeat the step with the same gains. Record the change in the response.

Table 2: *Tuning Log for Your Motor*

Run	K_p (V/°)	K_i (V/(°·s))	K_d (V·s/°)	t_r (s)	OS (%)	Final error (°)
1						
2						
3						

Run	K_p (V/°)	K_i (V/(°·s))	K_d (V·s/°)	t_r (s)	OS (%)	Final error (°)
4						
5						
6						
Final + mass						

Use your tuning log to answer the following questions.

1. Why does position control need derivative action more than the speed controller in the practical exercise, DC Motor PI Control: Manual Tuning?
2. What causes hunting around the setpoint, and how could it be reduced without removing integral action?
3. How did the added mass change the response, and which gain would you retune first?

4.0 Concluding Remarks

This practical exercise tuned a PID controller for the angle of a DC motor shaft by hand. Using MATLAB, Simulink and ACE-Lab hardware, the angle was measured from the encoder counts, and the proportional, derivative and integral gains were adjusted in turn using the measured response.

Proportional gain set the speed of the response, derivative gain damped the overshoot at the cost of amplifying measurement noise, and integral gain removed the final error caused by friction at the risk of hunting. Tuning in this order, one gain at a time, with each change linked to a measured effect, gives a reliable starting point for a controller that can later be refined with a model.

Recommended Citation

If you have used this practical exercise in your teaching, learning or research, please cite it using the reference below, adapting the format as required by your preferred referencing style.

Pickering, J. E. (2026). Manual PID Tuning of DC Motor Angle Control. ACE-Lab. Teaching Notes: ACE-Lab Practical Exercises, Version 1.0. Zenodo. https://zenodo.org/records/XXXXXXXX